Hoot: Project Breakdown

Predictive Household Inventory Tracker

Michael Hannan

Week 4 — September 17, 2026

Project Recap

- Households run on a small set of recurring consumables: milk, dish soap, toothpaste, paper towels.
- Manual shopping lists only work if someone remembers to update them before an item runs out.
- **Hoot's idea:** log purchases as they happen, learn each item's restock interval automatically, and surface a prediction — no list maintenance required.

Application Features — Core User Flows

- **Register an item** — name, category, unit, expected restock
- **Log a purchase** against an existing item — quantity, price, date
- **Dashboard** — tracked items ranked by how soon they're expected to run out
- **Add a one-time item** (e.g. a birthday candle) — shown as a plain checklist, excluded from prediction
- *(Optional)* Scan a barcode to auto-fill an item's name, category, and nutrition data

Application Features — Behind the Scenes

- **Interval regression:** on every new purchase, recompute the day-gaps between that item's past purchases and fit a linear regression to project the next purchase date
- **Initial fallback:** a brand-new item uses its manual `reorder_threshold` until at least 3 `purchase_log` entries exist
- **Prediction logging:** every regression run writes a row to `predictions` (`model_version`, `computed_at`) so accuracy can be measured later
- **Product Information:** barcode → check `product_lookups` cache → Open Food Facts / UPCitemdb → curated into `products` / `product_nutrition`

Task Breakdown

Category	Key Tasks
Database Design	Finalize ER diagram; implement core tables (`users`, `items`, `purchase_log`) in PostgreSQL via Entity Framework Core migrations; add extension tables (`products`, `product_nutrition`, `product_lookups`, `predictions`)
Backend / API	ASP.NET Core Web API controllers for item and purchase CRUD; DTOs for request/response shapes; Swagger/OpenAPI docs
Prediction Engine	Interval-extraction query; ordinary-least-squares regression; write results to `predictions`
Barcode Integration	`product_lookups` cache check; Open Food Facts client; UPCitemdb fallback client
Frontend	Static HTML/CSS pages + vanilla JavaScript (`app.js`) calling the Web API via `fetch`; dashboard, forms, restock progress bars
Testing & Deployment	Seed data and manual test pass on CRUD + prediction accuracy; optional Docker / Nginx deployment

Timeline for Tasks

Week / Date	Focus
Week 4 — Sep 17	Project breakdown (this deck); finalize schema design
Weeks 5–6 — Sep 18–Oct 1	Implement core 3-table PostgreSQL schema + EF Core models; basic ASP.NET Web API CRUD endpoints for items & purchases
Weeks 7–8 — Oct 2–21	Build dashboard frontend; implement interval-regression prediction pipeline + `predictions` logging
Week 9 — Oct 22	**Progress Report 1** — Done: schema, CRUD, basic dashboard. TODO: prediction refinement, barcode integration
Weeks 10–11 — Oct 23–Nov 9	Product catalog & barcode enrichment (`products`, `product_nutrition`, `product_lookups`); Open Food Facts / UPCitemdb clients
Week 12 — Nov 10	**Progress Report 2** — Done: prediction pipeline, barcode integration. TODO: polish, one-time items, testing
Weeks 13–15 — Nov 11–Dec 7	One-time item checklist flow; testing & bug fixes; optional Docker deploy; UI polish
Week 16 — Dec 8	Demo slides + demo video submitted
Week 16 — Dec 10	Final report + project code submitted

Application Views — Expected Screens

- **Login / Register** — create an account, sign in
- **Dashboard** — ranked list of tracked items with restock progress bars (main screen)
- **Item Detail** — one item's info plus its full purchase history; edit or delete
- **Add Item** — form to register a new tracked item (name, category, unit, threshold)
- **Log Purchase** — quick form to record a purchase against an existing item
- **One-Time Items** — checklist view for irregular, non-predicted items
- *(Stretch)* **Barcode Scan** — scan input that auto-fills item name/category/nutrition

All views are static HTML/CSS pages driven by `app.js`, which talks to the ASP.NET Web API over `fetch`.

Queries — What We Want to See From the Data

- **Dashboard list:** all items for a user joined to their latest prediction, ordered by `predicted_date` ascending *(list of records)*
- **Item detail:** one item's info plus its full `purchase_log` history, ordered by `purchase_date` *(single record)*
- **Regression input:** `purchase_log` rows for one `item_id`, ordered by `purchase_date`
- **One-time checklist:** items where `is_one_time = TRUE`
- **Prediction accuracy:** `predicted_date` vs. actual next purchase date, across `model_version`
- **Barcode lookup:** `products` joined to `product_nutrition` by `product_id`
- **Cache check:** `product_lookups` row for a barcode, before calling an external API

Actions — Manipulating the Data

- **INSERT** — new item; new `purchase_log` row on every purchase; new `predictions` row on every regression run; new `products` / `product_nutrition` / `product_lookups` rows on a barcode cache miss
- **UPDATE** — edit an item's fields (name, category, unit, `reorder_threshold`); correct a `purchase_log` entry (quantity, price, date)
- **DELETE** — remove a tracked item; remove a mistaken `purchase_log` entry
- Together with the queries above, this covers list retrieval and single-record retrieval